[Año]

Desarrollo web entorno servidor 2ºDAW

ESTUDIO TEMA 1

YOLANDA RODRÍGUEZ

ÍNDICE:

TEMA 1: DESARROLLO WEB EN ENTORNO SERVIDOR 2

EJERCICIO 1. 2

Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS. 3

Modelo de comunicaciones cliente – servidor y su relación con las aplicaciones web. 6

Estudio sobre los métodos de petición HTTP /HTTPS más utilizados. 7

Estudio sobre el concepto de URI (Idendificador de Recursos Uniforme)/URL/URN, estructura, utilidad y relación con el protocolo HTTP/HTTPS. 7

Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa. 8

Modelo de división funcional front-end / back-end para aplicaciones web. 8

Página web estática – página web dinámica – aplicación web – mashup 9

Componentes de una aplicación web. 10

Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor - lenguajes de programación utilizados en cada caso. 10

Lenguajes de programación utilizados en el lado servidor de una aplicación web (características y grado de implantación actual). 11

Características y posibilidades de desarrollo de una plataforma XAMPP. 12

1. **En que casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entrono de explotación. 12**

IDE más utilizados (características y grado de implantación actual). 13

Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual). 13

Apache HTTP vs Apache Tomcat..... 14

Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual). 15

Generadores de documentación HTML (PHPDoc): PHPDocumentor, ApiGen, … 15

Repositorios de software – sistemas de control de versiones: GIT , CVS, Subversion, ... 16

TEMA 1: DESARROLLO WEB EN ENTORNO SERVIDOR

Cuando hacemos referencia a desarrollo web en el entorno servidor , hablamos de la parte que consiste en crear la lógica “backend” que procesa peticiones, gestiona datos y genera respuestas dinámicas mediante lenguajes, frameworks y bases de datos especializados, como explican MDN y KeepCoding. A diferencia del frontend, que muestra la interfaz, el backend “piensa” y decide qué datos enviar según cada usuario o acción.Cuando una persona entra en una página web, su navegador realiza una petición al servidor, y este se encarga de procesarla y enviar una respuesta con la información que se debe mostrar.

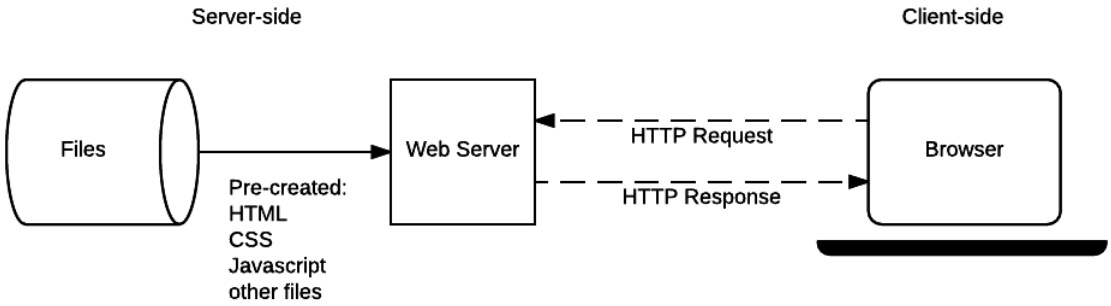
Esto permite crear páginas web que no sean siempre iguales, sino que puedan mostrar información diferente(dinámicas) dependiendo de los datos que se solicitn o de las acciones que realice el usuario. Por ejemplo, una aplicación puede consultar una base de datos, comprobar los datos de un usuario o guardar información y después enviar el resultado al navegador.Por tanto, en este tema veremos cómo se comunican el navegador y el servidor, los principales protocolos y métodos utilizados, las diferentes partes de una aplicación web y algunas de las herramientas y tecnologías que se utilizan para desarrollarlas.

Componentes esenciales del proceso:

- Recepción de la petición HTTP.
- Ejecución de lógica de negocio.
- Validación de datos.
- Acceso a bases de datos.
- Generación de la respuesta (HTML, JSON, etc.).

Conclusión:
El desarrollo web en entorno servidor es la base de cualquier aplicación web moderna, ya que gestiona datos, seguridad y lógica de negocio para ofrecer experiencias dinámicas y personalizadas.

Fuente utilizada: MDN Web Docs – Introducción al lado servidor



EJERCICIO 1.

1. Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS.

Los protocolos de comunicación son un conjunto de reglas que permiten que diferentes dispositivos puedan comunicarse y entenderse entre ellos cuando intercambian información.

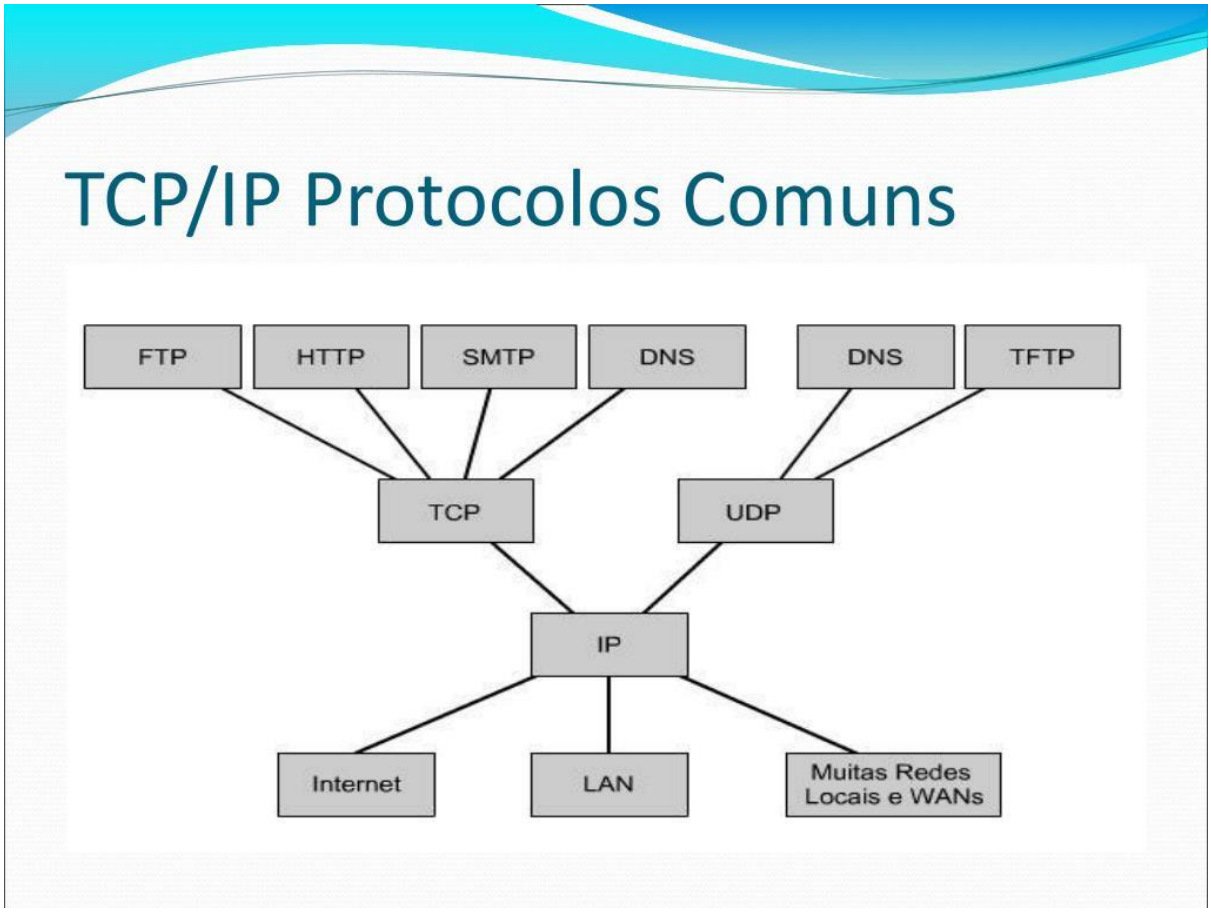
IP (Internet Protocol): es el encargado de identificar los dispositivos mediante direcciones" IP" y dirigir los paquetes de datos hasta su destino.

TCP (Transmission Control Protocol): controla el envío y orden de los paquetes.

HTTP (HyperText Transfer Protocol): permite la comunicación entre el navegador y el servidor web para solicitar y recibir páginas"html's" y otros recursos.

HTTPS (HyperText Transfer Protocol Secure): es HTTP actualizado y hecho mas seguro mediante una conexión TLS de tal forma que la información que se intercambia queda protegida.

Cuando navegamos por Internet, nuestro ordenador o móvil tiene que comunicarse con otros dispositivos para poder recibir la información que estamos buscando. Para que esta comunicación funcione correctamente existen diferentes protocolos, que son unas reglas que indican cómo se deben enviar y recibir los datos.



IP (Protocolo de Internet):
Es el protocolo que se encarga principalmente de direccionar y enviar los paquetes de datos hacia su destino. Cada dispositivo conectado a una red utiliza una dirección (IP) para poder ser identificado. Los datos que enviamos por Internet se dividen en pequeños paquetes y la IP le dice a los routers dónde tienen que dirigirlos. puede ser privada, (cuando identifica un dispositivo dentro de una red local, como la de una casa o una empresa), o pública(dirección utilizada para comunicarse con Internet). También puede ser estática, si mantiene la misma dirección, o dinámica, si puede cambiar en función de los distintos dispositivos que se puedan conectar dentro de una misma red.

TCP (Protocolo de Control de Transmisión):
Trabaja en equipo con IP. Mientras que la IP se encarga de enviar los paquetes al destino, TCP controla que la información se transmita correctamente. Por ejemplo, si una información se divide en varios paquetes, TCP ayuda a controlar su orden y a detectar si alguno no ha llegado correctamente para poder volver a solicitarlo.

Por eso se suele hablar conjuntamente de TCP/IP, ya que ambos protocolos trabajan juntos para permitir la comunicación entre dispositivos a través de Internet.Digamos que “monta y desmonta los trocitos de paquete”

HTTP (Protocolo de Transferencia de Hipertexto):
Es el protocolo utilizado para la comunicación entre un cliente(normalmente el navegador)y un servidor web. Cuando escribimos una dirección en el navegador o al pulsas sobre un enlace, el navegador realiza una petición al servidor y éste le devuelve una respuesta con la información solicitada. Esa información puede ser una página HTML, imágenes, vídeos, hojas de estilos u otros recursos.

Por ejemplo, cuando entramos en una página web el navegador puede enviar una petición GET al servidor para solicitar los recursos que necesita para mostrarla.

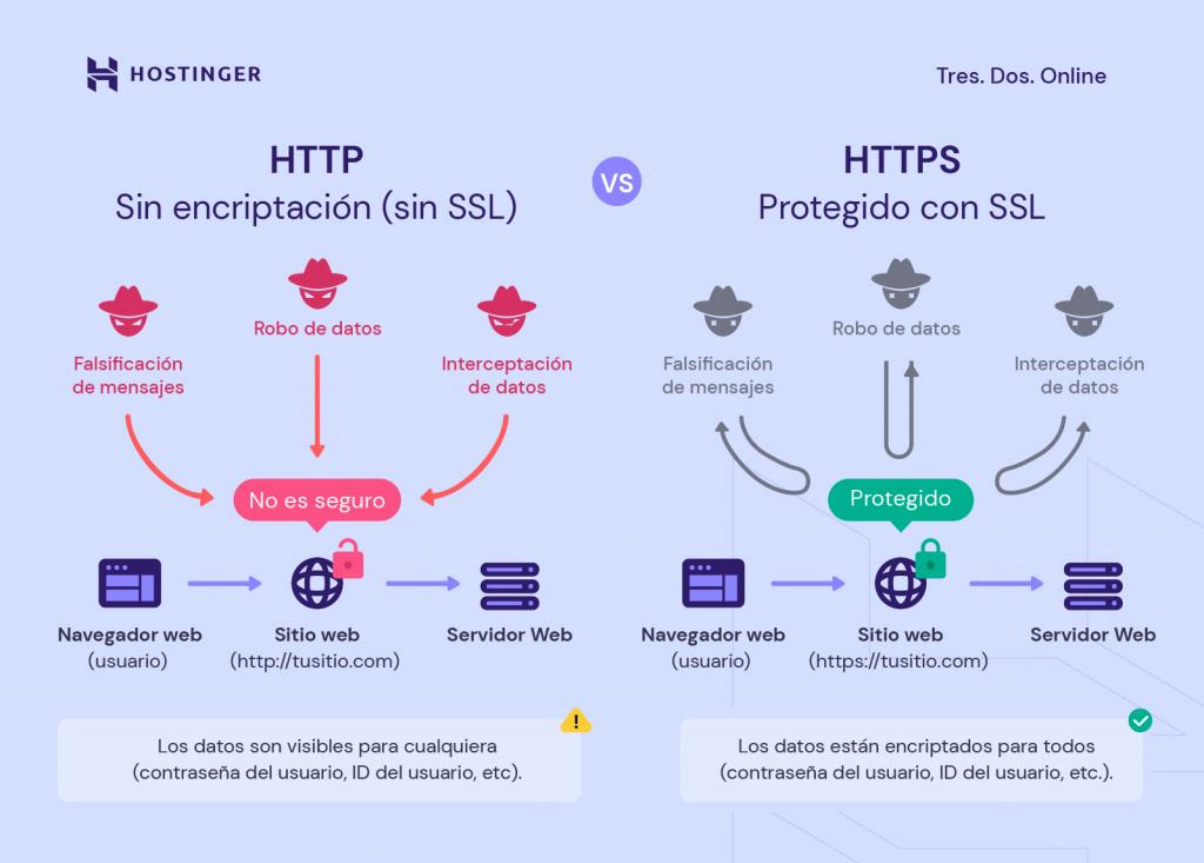
HTTPS (HTTP El seguro):
HTTPS funciona de forma parecida a HTTP pero añade una capa de seguridad mediante TLS. De esta manera, los datos que se intercambian entre el navegador y el servidor viajan cifrados y es más difícil que otra persona pueda leerlos por el camino si consigue interceptarlos.

actualmente es habitual ver https:// al principio de las direcciones de las páginas web. HTTP utiliza normalmente el puerto 80, mientras que HTTPS utiliza el puerto 443 y da un aviso de posible no seguro.

Una forma de entender como trabajan juntos es:

HTTP/HTTPS → TCP → IP

En resumen,cuando entramos en una página web, el navegador utiliza HTTP o HTTPS para solicitar la información. TCP se encarga de controlar la transmisión de los datos e IP se ocupa de dirigir los paquetes hacia el dispositivo correspondiente. En el caso de HTTPS, además, se utiliza TLS para proteger y cifrar la comunicación por el camino.



- **MDN Web Docs – Generalidades del protocolo HTTP:**
MDN – Generalidades del protocolo HTTP

- **Cloudflare – ¿Qué es el Protocolo de Internet (IP)?**
Cloudflare – Protocolo de Internet (IP)

- **Cloudflare – ¿Qué es TCP/IP?**
Cloudflare – TCP/IP

- **Cloudflare – ¿Qué es HTTPS?**
Cloudflare – HTTPS

2. Modelo de comunicaciones cliente – servidor y su relación con las aplicaciones web.

El **modelo cliente-servidor** es una forma de comunicación en la que el cliente realiza una petición y el servidor la recibe, la procesa y devuelve una respuesta. En una aplicación web, normalmente el cliente es el navegador y el servidor es el equipo que aloja la aplicación.Cuando entramos en una página web, el navegador actúa como **cliente** y realiza una petición al servidor mediante HTTP o HTTPS.

El servidor recibe esa petición, realiza las operaciones necesarias y devuelve una respuesta al navegador.

Por ejemplo, al entrar en una tienda online, el navegador puede pedir los datos de un producto.El servidor puede consultar esos datos en una base de datos y devolverlos para que el navegador los muestre. las aplicaciones web utilizan este modelo para permitir la comunicación entre el usuario y el servidor.

Fuente: MDN – Descripción general de cliente-servidor

3. Estudio sobre los métodos de petición HTTP /HTTPS más utilizados.

Los métodos HTTP indican al servidor qué acción queremos realizar sobre un recurso. Los más utilizados son GET, POST, PUT, PATCH y DELETE.

- **GET:** solicita información de un recurso. Por ejemplo, consultar una página o una lista de productos.
- **POST:** envía información al servidor y normalmente se utiliza para crear nuevos datos o enviar formularios.
- **PUT:** crea o sustituye un recurso completo.
- **PATCH:** modifica solamente una parte de un recurso.
- **DELETE:** elimina un recurso.

En una aplicación web, estos métodos permiten que el navegador y el servidor sepan qué operación se quiere realizar.

Fuente: MDN – Métodos de petición HTTP

4. Estudio sobre el concepto de URI (Idendificador de Recursos Uniforme)/URL/URN, estructura, utilidad y relación con el protocolo HTTP/HTTPS.

Un URI es un identificador que permite hacer referencia a un recurso. Dentro de los URI encontramos las URL, que indican dónde se encuentra un recurso, y las URN, que lo identifican mediante un nombre. El **URI (Identificador Uniforme de Recursos)** es el concepto general utilizado para identificar un recurso.

La **URL (Localizador Uniforme de Recursos)** indica dónde podemos encontrar ese recurso. Por ejemplo:

https://www.ejemplo.com/imagenes/foto.jpg

En ella podemos distinguir diferentes partes:

- https → protocolo.
- www.ejemplo.com → dominio.
- /imagenes/foto.jpg → recurso solicitado.

La **URN (Nombre Uniforme de Recursos)** identifica un recurso por su nombre, pero no indica necesariamente dónde se encuentra. Un ejemplo sería un ISBN utilizado para identificar un libro.

Las URL tienen una relación directa con HTTP y HTTPS porque el navegador las utiliza para indicar al servidor qué recurso quiere solicitar.

Fuentes:
MDN – URI
MDN – ¿Qué es una URL?

5. Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa.

Dividir una aplicación en capas permite organizar mejor el código y facilita su mantenimiento.

- Capa de presentación: es la parte que ve y utiliza el usuario.
- Capa de lógica o aplicación: contiene las operaciones y reglas que debe realizar la aplicación.
- Capa de datos: se encarga de almacenar y recuperar información, normalmente de una base de datos.

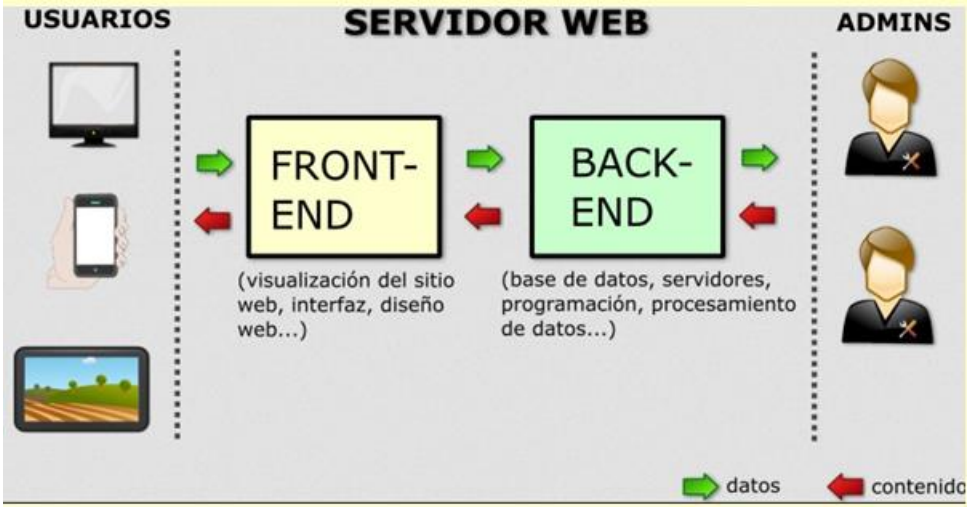
Las capas se comunican entre ellas para realizar las operaciones necesarias. Por ejemplo, el usuario solicita información desde la presentación, la lógica procesa la petición y la capa de datos consulta la base de datos.

Fuente: Microsoft Learn – Diseño de aplicaciones por capas

6. Modelo de división funcional front-end / back-end para aplicaciones web.

Se puede decir que en función de quien utilice la app y con qué objetivo o funcionalidad dividimos en dos los tipos de usuarios, usuario “normal”(autenticado o no) cuyo objetivo o función es la de usar o visualizar el sitio web o bien encargarse de la interfaz,diseño…. En este caso su funcionalidad es FRONT-END.

En el segundo caso tenemos al usuario encargado de administrar el contenido,los servidores,el código de programación,el proceso de datos….este usuario accede digamos a la parte que no se ve y su funcioinalidad es la de BACK-END.



front-end se ejecuta principalmente en el navegador y utiliza tecnologías como **HTML, CSS y JavaScript** para crear la interfaz con la que interactúa el usuario.

back-end funciona en el servidor. Se encarga, por ejemplo, de procesar datos, consultar una base de datos, controlar usuarios o generar respuestas para el navegador. Algunos lenguajes utilizados en esta parte son PHP, Python, Java o JavaScript mediante Node.js**.**

Fuente: MDN – La web y los estándares web

7. Página web estática – página web dinámica – aplicación web – mashup .

Una página **estática** muestra básicamente el contenido que está almacenado en sus archivos HTML. Todos los usuarios reciben el mismo contenido salvo que este se modifique.

Una página **dinámica** puede generar diferentes contenidos dependiendo de la petición. Por ejemplo, una tienda puede consultar una base de datos y mostrar un producto diferente según lo que solicite el usuario.

Una aplicación web va un paso más allá, ya que permite realizar acciones desde el navegador, como registrarse, comprar, consultar datos o enviar información.

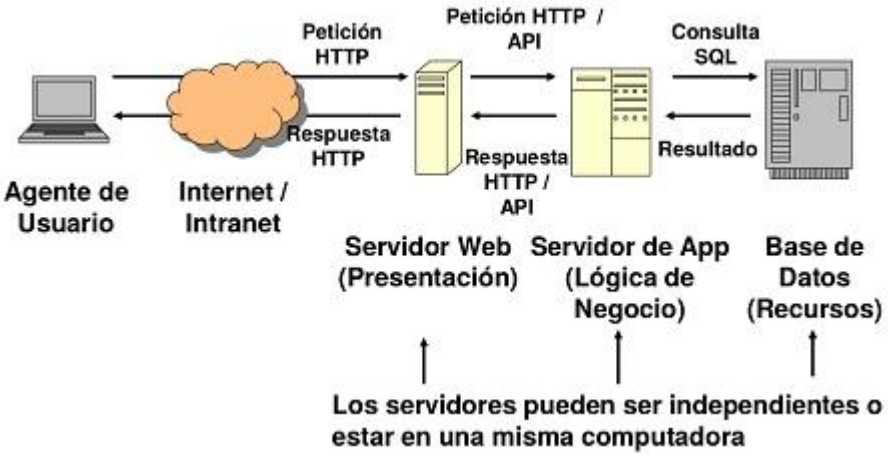
Un mashup combina datos o servicios de diferentes fuentes para crear una aplicación o servicio nuevo.

Fuente: MDN – Descripción general de cliente-servidor

8. Componentes de una aplicación web.

Una aplicación web puede estar formada por interfaz de usuario, servidor web, aplicación o lógica de negocio, base de datos y conexión de red. Los componentes pueden variar dependiendo de la aplicación, pero normalmente encontramos:

- **Cliente o navegador:** donde el usuario utiliza la aplicación.
- **Front-end:** muestra la interfaz y permite interactuar con ella.
- **Servidor web:** recibe las peticiones del navegador.
- **Back-end:** procesa las peticiones y realiza las operaciones necesarias.
- **Base de datos:** almacena la información.
- **Protocolos de comunicación:** permiten el intercambio de información, principalmente HTTP o HTTPS.



Todos estos componentes trabajan conjuntamente para que la aplicación pueda responder a las acciones del usuario.

Fuente: MDN – Cómo funciona la web

9. Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor - lenguajes de programación utilizados en cada caso.

Los programas del **lado cliente** se ejecutan principalmente en el navegador, mientras que los del **lado servidor** se ejecutan en el servidor antes de enviar el resultado al cliente.

En el lado cliente encontramos principalmente **HTML, CSS y JavaScript**. El navegador interpreta estos lenguajes y muestra la página al usuario.

En el lado servidor se pueden utilizar lenguajes como **PHP, Python, Java, C# o JavaScript con Node.js**. Estos programas pueden procesar información, consultar bases de datos y generar la respuesta que finalmente recibe el navegador.

Fuentes:
MDN – La web y los estándares web

10. Lenguajes de programación utilizados en el lado servidor de una aplicación web (características y grado de implantación actual).

Los lenguajes del lado servidor permiten realizar tareas que no se hacen directamente en el navegador, como acceder a una base de datos, comprobar usuarios o generar contenido dinámico.

PHP sigue teniendo una implantación muy importante en la web. Según W3Techs, en septiembre de 2026 se utiliza en el **69,9 % de los sitios web cuyo lenguaje del lado servidor es conocido**. Otros lenguajes muy utilizados son **Python, Java, C# y JavaScript con Node.js**, cada uno con diferentes frameworks y posibilidades de desarrollo.

Fuentes:
Manual oficial de PHP
W3Techs – Uso de PHP

11. Características y posibilidades de desarrollo de una plataforma XAMPP.

Es un entorno gratuito que permite instalar fácilmente diferentes herramientas necesarias para desarrollar aplicaciones web, entre ellas Apache, MariaDB, PHP y Perl.XAMPP facilita la creación de un entorno de desarrollo web en nuestro propio ordenador sin tener que instalar y configurar cada componente por separado.

Incluye principalmente:

- Apache: servidor web.
- MariaDB: sistema de gestión de bases de datos.
- PHP: lenguaje de programación del lado servidor.
- Perl: lenguaje de programación.
- También incluye herramientas como phpMyAdmin para trabajar con bases de datos.

Es especialmente útil para desarrollar y probar aplicaciones web de forma local antes de publicarlas en un servidor.

Fuente: Apache Friends – XAMPP

12. En que casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entrono de explotación.

La JVM (Java Virtual Machine) permite ejecutar programas Java. El JDK (Java Development Kit) contiene las herramientas necesarias para desarrollar y compilar programas Java.

Cuando estamos desarrollando una aplicación Java necesitamos el JDK**,** ya que incluye el compilador y otras herramientas necesarias para crear el programa.En el entorno de explotación, si únicamente queremos ejecutar una aplicación Java, necesitamos un entorno de ejecución compatible con esa aplicación y sobre todo, la JVM. No es necesario instalar el JDK completo si no vamos a desarrollar o utilizar sus herramientas.

La JVM es la encargada de ejecutar el código Java compilado, mientras que el JDK incluye las herramientas para desarrollar ese código.

Fuente: Documentación de Java – Oracle

13. IDE más utilizados (características y grado de implantación actual).

Un IDE (Entorno de Desarrollo Integrado) es un programa que reúne herramientas para programar, como editor de código, depurador, gestión de proyectos y otras funcionesLos IDE facilitan el desarrollo porque permiten realizar muchas tareas desde un mismo programa. Algunos de los más conocidos son:

- Visual Studio Code: ligero, gratuito y ampliable mediante extensiones.
- Visual Studio: muy utilizado para tecnologías de Microsoft y desarrollo .NET.
- IntelliJ IDEA: especialmente orientado a Java y Kotlin.
- Eclipse: muy utilizado en proyectos Java.

Según la encuesta de desarrolladores de Stack Overflow de 2025, Visual Studio Code continúa entre los entornos de desarrollo más utilizados, manteniendo una posición destacada. IntelliJ IDEA también aparece entre los IDE utilizados por los desarrolladore

14. Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual).

Un servidor HTTP/HTTPS es el programa encargado de recibir peticiones de los clientes y devolver los recursos solicitados. Entre los más utilizados están Cloudflare Server, Nginx y Apache HTTP Server.

Los servidores web reciben las peticiones realizadas por los navegadores mediante HTTP o HTTPS y devuelven páginas, imágenes, archivos u otros recursos.

Actualmente existen diferentes servidores web. Según los datos de W3Techs del 24 de septiembre de 2026, entre los sitios cuyo servidor web es conocido, Cloudflare Server representa el 30,9 %, Nginx el 30,8 % y Apache el 22,0 %.

Apache HTTP Server es un servidor de código abierto, configurable y compatible con diferentes sistemas operativos.

Fuentes:
W3Techs – Estadísticas de servidores web

15. Apache HTTP vs Apache Tomcat

Ambos son servidores web porque ambos tienen la capacidad de escuchar en un puerto (como el 80 o el 8080), recibir una petición HTTP de un navegador y devolver una respuesta HTTP. Sin embargo, la diferencia radical está en **su arquitectura interna y su especialización.**

Aunque Tomcat cumple la definición de servidor web, su verdadero poder y razón de ser es que es un **Contenedor de Servlets (Servlet Container)**.

- ➔ Apache HTTP Server se utiliza para servir páginas y recursos web y puede actuar también como proxy, además de admitir módulos para ampliar sus funciones. Es un servidor "nativo" (escrito en C). Está diseñado para ser un **maestro de la infraestructura de red**: gestiona miles de conexiones simultáneas de forma ultraeficiente, maneja seguridad a bajo nivel, reescribe URLs al vuelo (mod_rewrite) y comprime archivos a gran velocidad. Si le pides un archivo HTML, va al disco duro y te lo entrega de inmediato.
- ➔ Apache Tomcat está orientado a aplicaciones Java del lado servidor. Es capaz de ejecutar aplicaciones web Java y trabajar con tecnologías como Servlet y JSP.

  Implementa el protocolo HTTP: incluye un componente llamado **Connector (HTTP Connector)**. Este componente le permite actuar exactamente igual que cualquier servidor web básico: abre un socket, escucha peticiones de clientes y responde de forma directa sin necesidad de intermediarios.

  Puede servir HTML puro: Si pones un archivo index.html e imágenes dentro de la carpeta webapps/ROOT de Tomcat, las servirá perfectamente a cualquier usuario que lo solicite.
  Cuando le llega una petición para una página dinámica, Tomcat no solo busca un archivo; **activa la Máquina Virtual de Java (JVM)**, compila o ejecuta código, consulta bases de datos y genera el HTML en tiempo real.

Por tanto, no son exactamente alternativas directas: Apache HTTP Server está pensado como servidor web general y Tomcat como contenedor para aplicaciones Java web.

Fuentes:
Apache HTTP Server
Apache Tomcat – Introducción

16. Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual).

son programas que permiten acceder a páginas y aplicaciones web mediante protocolos como HTTP y HTTPS. Los más conocidos son Chrome, Safari, Edge, Firefox, Samsung Internet y Opera. El navegador actúa como cliente y se encarga de realizar peticiones al servidor y mostrar al usuario la información recibida.

Según StatCounter, en España, en agosto de 2026, los principales navegadores en el conjunto de dispositivos eran:

- Chrome: 74,6 %
- Safari: 13,96 %
- Edge: 3,05 %
- Samsung Internet: 2,86 %
- Firefox: 2,85 %
- Opera: 1,35 %

**Estos datos corresponden a España en agosto de 2026 por lo que pueden cambiaro variar con el tiempo.

17. Generadores de documentación HTML (PHPDoc): PHPDocumentor, ApiGen, …

Los generadores de documentación son herramientas que analizan el código y los comentarios realizados por el programador para crear automáticamente una documentación en HTML. En proyectos grandes es importante tener documentación para saber qué hace cada clase, función, método o parámetro.

phpDocumentor es una herramienta utilizada para generar documentación de proyectos PHP a partir del código y de los comentarios DocBlock. Puede generar documentación de la API y también diagramas de la estructura del código.

Otra herramienta que se ha utilizado para esta finalidad es ApiGen, orientada también a generar documentación de código PHP.

Estas herramientas permiten ahorrar tiempo y mantener la documentación relacionada directamente con el código del proyecto.

Fuente: Documentación oficial de phpDocumentor

18. Repositorios de software – sistemas de control de versiones: GIT , CVS, Subversion, ...